Methods

Methods I

The GeneralizedNotationNotation (GNN) method carries a plain-text model specification through source-processing transformations, bounded execution, and reporting. Parsing and code generation can be deterministic for fixed inputs and configuration; inference may involve stochastic sampling and depends on the runtime, dtype, and seed. The processing pipeline is organized into 25 numbered steps (steps 0–24), each implemented as a standalone module with a single responsibility. The methods below describe the path that a model travels from notation to executable cognitive model and back to analyzed results, and they state that path per model kind wherever the kinds part ways: the pipeline's plumbing is kind-agnostic, but rendering and execution are kind-aware by design. Every quantity reported in this section is produced from the live repository rather than asserted by hand; the closing subsection makes that contract explicit.

Parsing and Multi-Format Export

Parsing and Multi-Format Export I

The pipeline begins by ingesting GNN model files written in the plain-text notation. Parsing (step 3) reads each specification, builds an internal model representation, and re-emits it across a family of structured export formats so that downstream tools, and human readers, can consume the same model through whichever serialization they prefer. The corpus that exercises this stage spans 38 example model files organized into 12 curated corpora, and it spans the model kinds of sec. 3.2: minimal perception fixtures used to test the parser, discrete categorical agents, continuous linear-Gaussian state-space models, multi-agent swarms, hierarchical two-level compositions, and the scaling-study corpus that grows a single family across state-space sizes. The presence of hierarchical fixtures should be read as a coverage target, not as a claim that every current backend fully executes every hierarchical formulation; scaling hierarchical

active inference remains an active research problem in its own right (Rangarajan and Rao 2026). Treating parsing and export as a single round-trippable stage means a model authored once becomes immediately available as a typed object, a normalized text form, and machine-readable serializations without any manual re-encoding — and because the typed object carries the declared matrix vocabulary verbatim, the kind classification of sec. 3.2 is computed from the parse result rather than re-derived downstream.

Type Checking and Validation

Type Checking and Validation I

A model that parses is not yet a model that is well-formed. Steps 5 and 6 apply type checking and validation to confirm that the state spaces, observation modalities, control factors, and the matrices relating them are mutually consistent before any code is generated. This catches dimensional mismatches and malformed factor structures at the notation level, where the diagnostic is cheap and legible, rather than allowing them to surface as opaque runtime errors deep inside a numerical backend. Validation is where the kinds receive their shape contracts, and each kind's contract checks the objects its notation actually declares.

Type Checking and Validation I

For a discrete categorical specification, validation reads each declared `B` tensor through an orientation diagnostic: the canonical reading is column-stochastic, `B[next_state][previous_state][action]`, matching the pymdp convention (Heins et al. 2022) and the transition semantics fixed by eq. 3, and the check records every tensor's detected orientation in its receipt. A row-stochastic textbook literal — the transpose of the canonical layout — is flagged with the tensor, state factor, and flipped slice indices named, and the opt-in `--transpose-b` flag maps such literals onto the canonical order at load time instead of failing them, recording the transposition per tensor.

Type Checking and Validation I

For a continuous linear-Gaussian specification, validation enforces the Gaussian shape contract of eq. 9 and eq. 10: `prior_mean` must carry exactly as many entries as the latent state, `Q` and `prior_cov` must be square in the state dimension, and `R` must be square in the observation dimension. A covariance whose shape disagrees with the state or observation declaration is a validation-time diagnostic naming the mismatched matrix and its dimensions, not a broadcast error inside a generated script; and the optional closed-loop declarations are checked as a pair, since a `goal_mean` without a `control_gain` (or the reverse) is an incomplete control structure rather than a passive model.

Type Checking and Validation I

Validation supplies explicit checks of the declared model contract before rendering. Passing those checks establishes their scoped conditions, not a certificate of every later numerical or backend behavior; execution and analysis retain their own validation and source-bound receipts.

Rendering to Multiple Backends

Rendering to Multiple Backends I

The central act of the "Triple Play" is turning a validated specification into executable cognitive models. The rendering stage emits backend-specific code for 12 registered target frameworks — PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn — so that a single GNN model can be instantiated as a simulation in whichever computational ecosystem a researcher already works in. Each backend is addressed through a registry key that maps the abstract model onto that framework's idioms for representing generative models and performing inference. The full mapping from registry key to backend is given below.

Rendering to Multiple Backends I

Table 1: Render targets declared in `src/gnn/render/framework_registry.py`, one row per registry key in registry order. The *Executes* column is the registry's own `supports_execution` flag: 12 of 12 backends have a Step-12 executor, and a `render-only` entry would mean generated code with no Step-12 executor. Read this table against tbl. 5, which shows which families actually target each backend.

Registry key	Backend	Executes
`pymdp`	PyMDP	yes
`rxinfer`	RxInfer.jl	yes
`activeinference_jl`	ActiveInference.jl	yes
`jax`	JAX	yes
`discopy`	DisCoPy	yes
`pytorch`	PyTorch	yes
`numpyro`	NumPyro	yes
`stan`	Stan	yes
`bnlearn`	bnlearn	yes
`cpomdp`	cpomdp	yes
`thrml`	THRML	yes
`ngclearn`	ngc-learn	yes

Rendering to Multiple Backends I

Because rendering is kind-aware, the registry is best read as a capability matrix rather than a flat list: which backends can render a discrete categorical model, which can render a continuous linear-Gaussian model, and which have execution adapters at Step 12. That per-framework, per-kind view is auto-injected below.

Rendering to Multiple Backends I

Table 2: Model kinds each render framework supports, read entirely from the flags in `src/gnn/render/framework_registry.py` — the same source as tbl. 1. A continuous cell of `unsupported` is the renderer's own status for continuous-state models; continuous-only targets do not render categorical models. An executor entry declares dispatch support, not acceptance of every model or inference mode; a render-only entry has no Step-12 executor. Read with tbl. 1, which groups the same flags by model kind.

Framework	Discrete render	Continuous render	Executor status
PyMDP	yes	unsupported	executor
RxInfer.jl	yes	yes	executor
ActiveInference.jl	yes	unsupported	executor
JAX	yes	yes	executor
DisCoPy	yes	unsupported	executor
PyTorch	yes	yes	executor
NumPyro	yes	yes	executor
Stan	yes	yes	executor
bnlearn	yes	unsupported	executor
cpomdp	no	yes	executor
THRML	yes	unsupported	executor
ngc-learn	no	yes	executor

Rendering to Multiple Backends I

The registry surface of these backends — registry key, display name, implementation language, and which of them the cross-framework gate compares — is summarized in fig. 1; the family-by-backend coverage itself is shown in fig. 6.

Rendering to Multiple Backends I

GNN Rendering Backend Registry
cross-framework reference family: continuous

Registry Key	Display Name	Language	Render Subdir	Cross-Framework
pymdp	PyMDP	Python	yes	
rxinfer	RxInfer.jl	Julia	yes	profiled
activeinference_jl	ActiveInference.jl	Julia	yes	
jax	JAX	Python	yes	profiled
discopy	DisCoPy	Python	yes	
pytorch	PyTorch	Python	yes	
numpyro	NumPyro	Python	yes	profiled
stan	Stan	Stan	yes	declared
bnlearn	bnlearn	Python	yes	
cpomdp	cpomdp	Python	yes	
thrml	THRML	Python	yes	
ngclearn	ngc-learn	Python	yes	declared

12 registered backends · 12 with a render-output subdir under src/gnn/render/ · highlighted 3 backends are profiled in the continuous reference comparison
stan, ngclearn declared by the family but outside MAINTAINED_FRAMEWORKS, so never profiled

Figure 1: The GNN rendering backend registry as a table: one row per registered backend, with its registry key, display name, implementation language, whether a render-output subdirectory exists under `src/gnn/render/`, and its role in the cross-framework reference comparison. Amber rows are the backends the reliability gate profiles for the continuous family; the tan row (Stan, ngc-learn) is declared by that family but sits outside the maintained set, so it is never profiled; a red Render-Subdir cell would mean a registry entry with no subdirectory. The table is read directly from `src/gnn/render/framework_registry.py` and the model-family manifest at commit 609a35f3d, and its footer summarizes the row counts. Take-away: registration, render presence, and profiling status are three different claims, and the table keeps them apart.

Discrete categorical models use the categorical-capable renderers, subject to each renderer's dimension, parameter, and composition checks. The continuous-only cpomdp and ngc-learn targets report these models as `unsupported`. Experimental THRML must be selected explicitly: it implements fixed-action categorical posterior sampling, with bounded independent or validated joint compositions, and does not optimize policies or expected free energy. Registry membership therefore does not imply categorical support for every target or every categorical composition; the profiled cross-framework reference in sec. 5 is a separate, selected comparison.

Continuous linear-Gaussian models take the native paths built for them. The RxInfer.jl renderer carries dedicated continuous strategies that emit reactive message-passing code over the Gaussian state-space model; the JAX, PyTorch, and NumPyro renderers generate standalone scripts that filter the belief state numerically — and, when the closed-loop declarations of eq. 11 are present, steer it toward the preferred state — with the JAX lane enabling 64-bit precision for the covariance arithmetic; and the Stan renderer emits the model as a probabilistic program. The categorical-only backends — pymdp, ActiveInference.jl, DisCoPy, and bnlearn — do not fabricate a translation: each reports the continuous model as `unsupported`, with a recorded reason, and the status is counted under unsupported framework renderings rather than as a failure — never rendered, never executed, and excluded from the success

denominator so the run's rate measures the backends that could actually attempt the model. DisCoPy's boundary is representative: its translator draws categorical POMDP string diagrams and has no linear-Gaussian diagram semantics, so a continuous model is reported unsupported rather than drawn as a discrete stand-in.

Composed kinds require admission of the whole declared composition, not just one recognized substrate. Supported categorical multi-agent paths map per-agent declarations to their target structures. Independent continuous agents use native JAX or RxInfer.jl paths only when independence and a complete linear-Gaussian parameter block for every agent are declared; other continuous-agent compositions are reported `unsupported`. Generic hierarchical paths without a semantic execution contract may compose per-level matrices into a joint POMDP or use the RxInfer.jl two-level strategies; the maintained

`block_reset_v1` and `timed_soft_controller_v1` fixtures instead route to structure-preserving JAX execution and are explicitly unsupported elsewhere; parameter-learning paths use strategies for the declared Dirichlet matrix. These paths retain backend-specific capability boundaries, and a recognized component does not authorize silently dropping another declared component.

Structural wrappers have no renderable form at all: they declare boundary structure only, with neither a categorical nor a continuous parameterization, so every framework reports them `unsupported` upstream with the structural-spec reason rather than silently rendering them as a discrete stand-in that would fail on missing matrices. The status is the design working, not a defect: a wrapper is an interface statement, and the pipeline's response is to say so in every receipt rather than to guess a model body the author never

declared.

By generating code rather than asking authors to port models by hand, the method keeps a single source of truth in the notation while still reaching the discrete message-passing libraries (Heins et al. 2022), reactive message-passing lineage (Bagaev and Vries 2021), and categorical, diagrammatic frameworks (Felice et al. 2021) that different research communities have built. The underlying mathematics that compatible backends share — inference over the generative model the notation declares, whether that is categorical free-energy minimization (Da Costa et al. 2020; Smith et al. 2022) or Gaussian filtering and control — is the common substrate the renderer attempts to preserve; where a backend cannot express a kind faithfully, GNN records that boundary as an explicit unsupported status rather than treating all registered backends as equivalent.

Execution

Executing by Model Kind I

Generated backend code is not left as a static artifact. The execution stage (step 12) runs the rendered models, driving the inference and behavior that the notation describes and producing concrete traces, beliefs, and outputs. The 12 render targets — PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn — have implemented execution adapters at this step. This is a registry claim, not evidence that every adapter has run in a given environment. Model-kind admission, validated parameters, package and toolchain readiness, supported Python, and remaining budgets determine whether an execution can proceed; the current-run receipt records its actual outcome.

Executing by Model Kind I

Discrete executors run the categorical models end to end, and they guard their own runtime assumptions: the generated pymdp scripts, for example, sanity-check the installed library for the current Agent interface at startup and exit fast with a clear message when an unsupported wheel is present, so a stale environment produces a legible diagnostic instead of a mysterious failure deep in a simulation loop.

Continuous executors run the generated standalone scripts over the Gaussian state-space model — filtering to a posterior, and with the closed-loop declarations in place, closing the control loop on beliefs — so a continuous model produces traces on the same terms as a discrete one, on the backends that carry the kind.

Generator-backed and render-adjacent lanes extend execution beyond the registry's render paths. The bnlearn lane is generator-backed: it renders a runnable Python program from the same parsed model the other lanes consume, and it skips with a recorded status when the `bnlearn` package is absent, so a lean checkout degrades explicitly rather than failing. A separate execute-side bridge verifies rendered Lean documents against the fep_lean bridge contract, which is how the notation reaches a proof assistant without a render-registry entry. Skipped, degraded, and unsupported lanes are recorded as explicit statuses in the cross-framework comparison rather than silently omitted.

Executing by Model Kind I

For a successfully executed, source-bound model, this closes the loop from text to numerical traces. Claims about its behavior require the actual result, inference mode, environment, and validation receipts; a generated script or implemented adapter alone does not establish that behavior.

Long-Running Orchestration Contracts

Long-Running Orchestration Contracts I

GNN 3.0.0 extended the pipeline with three safe-by-design orchestration contracts that let an extended run be observed, paused, resumed, and audited without ever mutating live infrastructure. *Durable observation streams* record file- and array-backed stream manifests with content checksums and replayable execution traces, so that a long run can be re-derived deterministically before any live sensor or device-backed stream is introduced. *Resumable run sessions* carry run-session manifests with atomic checkpoint and resume, status inspection, and cancellation-safe cleanup, so that an extended model-family acceptance run can be interrupted and resumed without corrupting partial state. *Auditable container plans* generate hardened container plans together with an explicit static security review and a rollback descriptor, deliberately stopping short of touching any real cluster. Each contract only generates and validates

data; none performs live mutation, and each is governed by a strict acceptance gate and exercised through dedicated Model Context Protocol tools. fig. 2 shows how the three contracts compose into one inspectable orchestration surface.

Long-Running Orchestration Contracts I

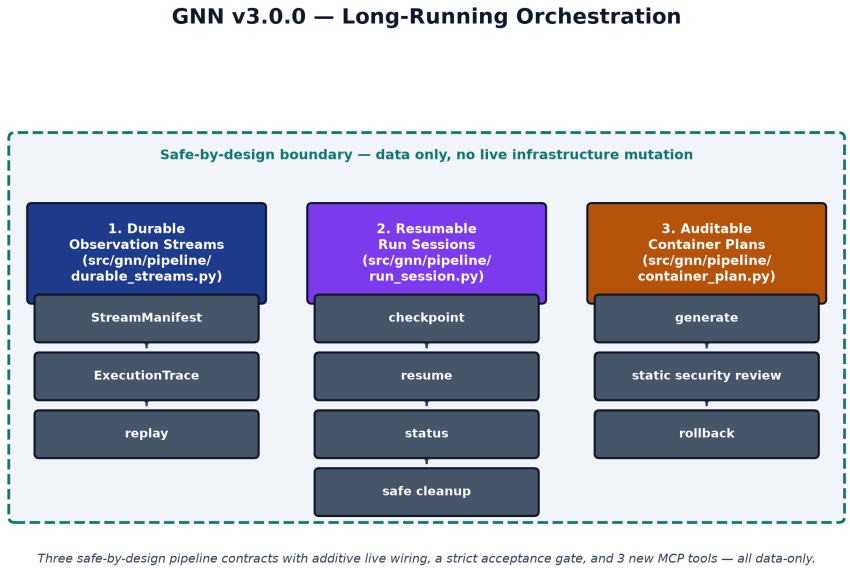


Figure 2: The three safe-by-design orchestration contracts introduced in GNN 3.0.0, side by side inside the dashed boundary that states their shared invariant: generate and validate data only, never mutate live infrastructure. Each column names the contract's source module under `src/gnn/pipeline/` and chains its stages top to bottom — StreamManifest, ExecutionTrace, and replay for durable observation streams; checkpoint, resume, status, and safe cleanup for resumable run sessions; generate, static security review, and rollback for container plans. Arrows give the order the stages run in, and nothing exits the boundary. The panel is a fixed-layout drawing of the three contracts; the acceptance gates and the Model Context Protocol tools that exercise them are described in the text above. Take-away: long-running orchestration gains capability only as fast as it becomes inspectable.

Analysis, Visualization, and Reporting

The final group of stages turns execution results into interpretable evidence. Analysis (step 16) processes the outputs of execution into structured findings; visualization (step 8) and the rendering of figures (step 9) translate model structure and results into graphical form, supporting the graphical leg of the Triple Play; and the reporting stage (step 23) assembles these artifacts into a coherent summary of what the model is and how it behaved. Known output locations and current-run identity indexes connect a notation file to its report artifacts. Tracing a figure or number requires its selected model identity and source, script, result, and artifact receipts; an output path alone is insufficient evidence of a current run. Analysis reads the per-kind rendering and execution receipts as well: a continuous model whose categorical-only lanes report `unsupported` yields a different

evidence profile than one whose continuous-capable lanes failed, and the reports keep those situations apart.

Reproducibility and Auto-Injection

Every count reported in this manuscript is emitted by the producer `scripts/z_generate_manuscript_variables.py` from the repository state at commit 609a35f3d, not typed by hand. The producer reads that commit's source surfaces directly — the step modules, the source tree, the model-family manifest, and the framework registry — together with the project's maintained ledgers, notably the Model Context Protocol tool audit (`src/gnn/mcp/audit_report.json`, itself regenerated by the test suite). From these it emits a manifest of named tokens — counts of pipeline steps and their modules, source packages and files, model families, backends, example files partitioned by model kind, tests, and tools — plus the two capability tables this manuscript injects: the model-kind table (tbl. 1) and the framework capability matrix (tbl. 2). The renderer substitutes the

tokens into the prose at build time. Filesystem-derived counts are recomputed on every run; ledger-derived counts (such as the backend registry) are re-read from a fixed repository state, so a fixed repository state yields the same values, and authors are prohibited from hard-coding any number that has a corresponding token. This makes the manuscript a faithful, regenerable description of the system it documents: when the 25-step pipeline, the 12 rendering backends, or the 38-file example corpus change — or when the discrete-to-continuous exemplar balance shifts — the reported numbers change with them on the next build, and `scripts/check_manuscript_tokens.py` fails the build if a section reintroduces a literal that a token already owns.